

Recoil — Technical Case Study

System-Level Safety and Forensic Protection for Human Rights Defenders on Linux

Author: Ali Zain, Independent Systems Developer

Contact: alizain.arch@gmail.com

Repository: <https://github.com/Zainium-Dynamics/Recoil>

Website: <https://zainiumdynamics.tech/projects#recoil>

License: GNU General Public License v3.0 **Date:** May 2026

The Problem in Context

In repressive environments, a journalist's laptop is not just a work tool — it is an archive of sources, evidence, and communications that may be irreplaceable and that adversaries have strong incentives to destroy or corrupt. When a device is seized and returned, the user has no reliable way to determine whether files were modified, deleted, or whether surveillance software was installed during the period of confiscation. When malware is deployed through a phishing attack, it typically deletes its tracks and any evidence it was sent to destroy before the user even knows it was there. When a terminal command goes wrong under the stress of a deadline or an emergency, there is no undo.

These are not hypothetical scenarios. They are documented patterns in the threat landscape faced by journalists, human rights defenders, and activists operating under state-level adversarial pressure. The operational security guidance that exists today tells these users to use full-disk encryption, to use Signal for communications, and to use Tor for browsing. It has almost nothing to say about what happens when the files on the device itself are deleted or modified — because until now, there has been no practical tool to address that problem on Linux.

Recoil was built to fill that gap.

What Recoil Does

Recoil operates across three simultaneous layers of protection that work together automatically after a single setup command.

The first layer is the shadow mirror. At setup, Recoil creates a complete copy of the device's entire Linux filesystem inside a hidden directory — on Debian this is `/.recoil-debian/`, on Ubuntu `/.recoil-ubuntu/`, on other distributions it follows the same pattern. The copy uses hard links or copy-on-write operations depending on the filesystem, which means it consumes essentially no additional disk space. The entire directory is then locked with a Linux kernel-level flag called `chattr +i`. This flag is enforced by the operating system kernel itself, not by permissions. Once it is set, not even the root user can delete or modify the contents of the shadow directory through any standard command. `sudo rm -rf /.recoil-debian/` returns "Operation not permitted." Malware running as root cannot delete the mirror. A state actor with physical access cannot wipe it without specialised forensic tools. If the device's own filesystem is corrupted or deleted, the mirror survives and the system can be fully restored from it.

The second layer is real-time interception. Recoil runs two interception mechanisms simultaneously. The first is an `LD_PRELOAD` shared library that intercepts destructive operations — file deletion, overwriting, renaming — for all standard programs the moment they execute. The second is an eBPF layer that operates inside the Linux kernel itself and catches the same operations for any program that might bypass the first layer, including statically compiled malware tools that are specifically engineered to evade `LD_PRELOAD`-based interception. When either layer detects a destructive operation, the affected file is immediately copied into an encrypted vault in the background before the operation proceeds. The user sees no interruption. The file appears deleted. But it is recoverable.

The third layer is the forensic chronology. Every significant event on the system is recorded in a tamper-proof database: every file downloaded and its source URL and cryptographic hash, every software package installed and the repository it came from, every git repository cloned and its remote URL and exact commit, every build command executed with the compiler version and flags, every file deleted and the process that deleted it. The audit log is structured as a Merkle tree — a cryptographic data structure where any modification to any historical record changes a root hash that can be independently verified. If the log is tampered with, recoil verify detects it immediately. This gives users cryptographically verifiable evidence of their system's state at any point in time.

How This Serves Human Rights Defenders Specifically

Evidence preservation against malware. A journalist investigating government corruption receives a spear-phishing document. The malware it contains executes, exfiltrates files, and then deletes itself along with the evidence files it was targeting. On a standard Linux system, those files are gone. On a system protected by Recoil, every deleted file was intercepted before deletion and is sitting in the encrypted vault. The journalist runs recoil restore and recovers the files. The malware's deletion operation accomplished nothing.

Device tampering detection after seizure. An activist's laptop is confiscated at a border crossing and returned two days later. With Recoil, the activist can run recoil verify to check the Merkle tree integrity of the audit log and recoil history --since 48h to see every file that was accessed, modified, or deleted during the period of confiscation. If surveillance software was installed, it appears in the chronology. If files were accessed or copied, that access is recorded. If anything was changed, the Merkle tree verification fails and Recoil reports exactly which files were affected.

Accidental deletion under pressure. A human rights defender working under extreme time pressure to publish a report before an imminent arrest misruns a terminal command and accidentally deletes two weeks of interview notes. The files were in the vault. They run recoil restore ~/notes and have everything back in seconds.

Legal and evidentiary use. The Merkle tree audit log provides cryptographically verifiable proof that specific files existed on a device at a specific time with specific contents. This has direct evidentiary value in legal proceedings and in human rights documentation submitted to international bodies. The log cannot be retroactively forged without breaking the Merkle root verification — which any independent party can check.

Digital security trainers. Frontline Defenders, Access Now's Digital Security Helpline, and similar organisations deploy and support Linux devices for high-risk users. Currently, giving those users meaningful protection against the file deletion and tampering threat requires expert configuration of multiple separate tools — file integrity monitors, encrypted backup systems, audit frameworks — that are individually complex and collectively impractical for users who are not technical. Recoil replaces all of that with a single sudo recoil setup command and zero ongoing configuration requirement.

Architecture

The shadow directory is named after the detected Linux distribution and carries a leading dot, making it invisible in standard file listings. On seizure, an adversary conducting a casual filesystem search will not find it. It is protected at the kernel level, not by obscurity — but the combination of kernel-level immutability and reduced discoverability provides meaningful defence in depth.

```
/.recoil-zainium/          Hidden. Kernel-immutable.
|
|— root-mirror/           Complete filesystem mirror
|  |— bin/ sbin/ lib/ lib64/ Real directories — not symlinks
|  |— usr/ boot/ etc/ root/
|  |— home/ var/ opt/ srv/
|
|— recoil-b/
|  └─ recoil-ctl          Works even if /usr/bin is deleted
|— vault/                Encrypted deleted-file storage
|— logs/                 Merkle HMAC-chained audit trail
|— db/                   AES-256-GCM forensic chronology
└─ .config               AES-256-GCM encrypted config
```

bin/, sbin/, lib/, and lib64/ are always created as real directories, not symlinks, even on systems where the live filesystem uses symlinks for these paths. This ensures that recovery works completely even if the directory structure of the live system has been damaged or deliberately corrupted.

The recoil-ctl static binary at /.recoil-debian/recoil-b/recoil-ctl has zero runtime dependencies. It was compiled against musl libc and requires no dynamic linker, no shared libraries, and no working standard system paths. If an adversary has deleted everything in /usr/bin and /lib, this binary still executes and can restore the complete system.

Cryptographic Design

All sensitive data is encrypted with AES-256-GCM authenticated encryption. A fresh random nonce is generated for every single encryption operation, making nonce reuse structurally impossible. The authentication tag on every encrypted object detects any modification — including single-bit changes — before any plaintext is returned.

Key derivation uses Argon2id with 64 megabytes of memory cost, three iterations, and four-way parallelism. These parameters mean that an adversary attempting to crack the master password by brute force needs to allocate 64 megabytes of memory for every single attempt. On current hardware, this limits an attacker to a few hundred attempts per second at most — making dictionary attacks against a reasonably strong password computationally infeasible even with significant resources.

The brute-force protection system stores its state on disk using Unix wall-clock timestamps. It locks for 20 minutes after three consecutive failures, for three hours after 15 failures within any rolling one-hour window, and permanently after 50 failures within any rolling 24-hour window. These lockouts survive device reboots. An attacker cannot reset the lockout counter by simply powering the device off and on.

The audit log uses a Merkle tree structure. Every deletion event is a leaf node. Internal nodes are computed as $\text{SHA-256}(\text{left_child} \parallel \text{right_child})$. The Merkle root is stored in an HMAC-protected file inside the immutable shadow layer. Every call to recoil verify recomputes the root from the leaves and compares it against the stored root. Any modification, truncation, or insertion into the audit log produces a different root, which the verification immediately detects and reports as a security event — triggering a lockdown of vault operations until an administrator explicitly reviews and clears the alert.

The recoil-ctl Emergency Recovery Binary

The static emergency binary at `/.recoil-zainium/recoil-b/recoil-ctl` is the tool of last resort. It accepts the same commands as the main recoil binary: `restore --system` for full filesystem restoration, `emergency-restore <path>` for a specific path, `verify` for integrity checking, `status` for a health report, and `unlock` for authenticated access to specific shadow layer paths. Every command requires the master password, which means an adversary who has the device but not the password cannot use the recovery binary to access vault contents or modify the shadow layer.

This binary exists specifically for the scenario where the live system has been so severely damaged — by malware, by a failed update, by deliberate sabotage, or by a catastrophically mistaken terminal command — that no standard recovery tools are available. As long as the device can boot to any minimal shell environment, this binary can restore the complete system.

Privacy Properties

Recoil makes no network connections under any circumstances. It has no update mechanism, no telemetry, no registration, and no remote reporting. Every piece of data it stores — vault contents, audit log entries, chronology records, configuration — lives on the local device, encrypted with a password only the user knows. There are no third parties who can be compelled by court order or government demand to provide access to a user's Recoil data, because no third parties have any access to it.

Six-Phase Development Plan

Phase 1 — Foundation and Security Core builds the complete application framework, OS and filesystem detection, the AES-256-GCM encryption pipeline, the PBKDF2 key derivation system, and the brute-force protection mechanism. Currently under active development with 25 tests passing.

Phase 2 — Immutable Filesystem Mirror creates the shadow directory, mirrors the complete filesystem, applies chattr +i kernel-level protection, deploys the static recovery binary, and implements self-relocation from user home directories to system paths. Currently under active development.

Phase 3 — Encrypted Vault, Argon2id, and Merkle Audit Log implements per-file encrypted vault storage with asynchronous workers for zero-latency interception, upgrades key derivation to Argon2id, and builds the Merkle tree audit log with tamper detection. Independent cryptographic review and fuzzing campaign included. NLnet proof-of-concept milestone equivalent.

Phase 4 — Dual-Layer Interception and Forensic Chronology deploys the LD_PRELOAD and eBPF interception layers simultaneously for complete coverage, and builds the

forensic chronology engine recording all download, installation, clone, build, and deletion events.

Phase 5 — Recovery Tools, Daemon, and Distribution Packages implements recoil restore and all vault commands, deploys the system daemon with systemd integration, and publishes distribution packages for Debian, Fedora, and Arch Linux.

Phase 6 — Validation and Final Release runs adversarial testing across six Linux distributions, begins the Redox OS port, publishes the validation report, releases v1.0.0, and publishes standalone reusable libraries for other security tool developers.

Related Projects and Collaboration Opportunities

Frontline Defenders provides direct protection to human rights defenders globally and has an established programme of deploying and supporting Linux devices for high-risk users. Engagement with their technical team during Phase 5 would help ensure that Recoil's default configuration and user-facing error messages are appropriate for users who are not technical specialists.

Access Now's Digital Security Helpline provides emergency digital security support to civil society. Their team regularly works with users whose devices have been compromised or seized. The forensic capability of recoil verify and recoil history is directly relevant to the incident response workflows they support, and their experience would be valuable input for the Phase 6 validation process.

EFF's Surveillance Self-Defense project maintains guides for at-risk users. Recoil addresses a gap in their current Linux guidance — there is currently no simple recommendation for Linux users who want protection against file deletion by malware or physical access attacks. A validated v1.0.0 release would be a candidate for inclusion in their guidance.

Future Development

Beyond v1.0.0, Recoil's portability roadmap extends to FreeBSD and OpenBSD, which are used in high-security environments by some human rights defenders and privacy-focused organisations. Container environment support would extend the tool's reach to server

deployments and cloud-hosted journalism infrastructure. A standalone recoil-core library will allow other open-source security tools to incorporate Recoil's vault and audit log components without depending on the full binary. These directions are not part of the current funding request but reflect a long-term commitment to the project.